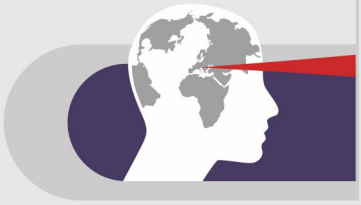


SMARTEH®
LIVING SYSTEMS

用户手册

OpenPLC Editor

版本 4

SMARTEH d.o.o. / Poljubinj 114 / 5220 Tolmin / Slovenia / Tel.: +386 (0)5 388 44 00 / e-mail: info@smarteh.si / www.smarteh.si

撰写：SMARTEH d.o.o.

用户手册

Index

OpenPLC Editor / LPC Manager 用户手册

1 关于本文档

LPC Manager 用户手册说明如何使用应用程序 LPC Manager。

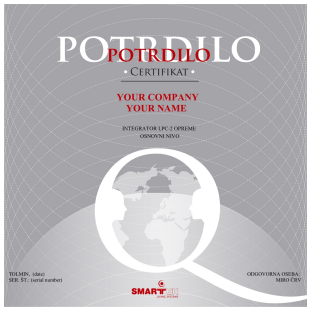
1.1 谁应阅读本文档

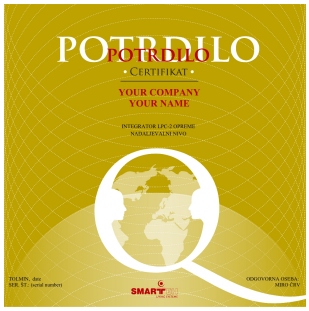
如果您是 LPC Manager 软件的新用户，希望入门使用，或正从先前版本升级，则应阅读本文档。

在该短期课程中，您将学习全部基础内容，并有机会成为经认证的 SMARTEH d.o.o. 集成商。您不仅能抢先掌握要点，还将加入不断壮大的 LPC 用户群体，有机会直接向专家提问，并与世界各地的其他集成商及公司建立联系……

注意：由于 LPC Manager 基于 IEC 61131-3 国际标准，更详细的信息请参阅 PLC（可编程逻辑控制器）编程语言国际标准 IEC 61131-3。

如需了解更多信息，请致电 +386 5 388 44 00，或发送电子邮件至 info@smarteh.si，预约 LPC 认证培训名额。





1.2 Terminology

本手册通篇使用多种表述。以下为其中部分术语的说明。

1.2.1 LPC Family products based terminology

LONGO™

... is a family of products (hardware and software) and is a trademark of SMARTEH d.o.o.

LPC™

... Longo Programming Controller and is a trademark of SMARTEH d.o.o.

IDE

⋯ is a integrated development environment.

LPC-2 Programming Controller

... is a family of hardware modules (MCU module, I/O modules and other modules).

LPC Smarteh IDE, LPC Manager

... are members of LPC family software.

1.2.2 LPC Manager based terminology

Beremiz

⋯ is a free software framework for automation (http://www.beremiz.org)

IEC 61131-3

⋯ is an international standard for programmable controllers, programmable languages

PLC

⋯ Programmable Logic Controller

MCU

⋯ Main Control Unit

POU

⋯ Program Organization Unit

编程语言

- IL
- ST
- LD
- FBD
- SFC

True

⋯ Logical 1, on, active, high state

False

⋯ Logical 0, off, not active, low state

1.2.3 Conventions used in this document

Items appearing in this document are sometimes given a special appearance to set them apart from the regular text. Here's how they look:

Italic
Used for marking important keywords.

NEW:
Used to mark sections which changed most from previous versions.

2 LPC MANAGER 软件

2.1 简介

LPC Manager 软件（LONGO Programmable Controller Manager software）用于对 Smarteh 控制器的 LPC-2 系列进行编程。

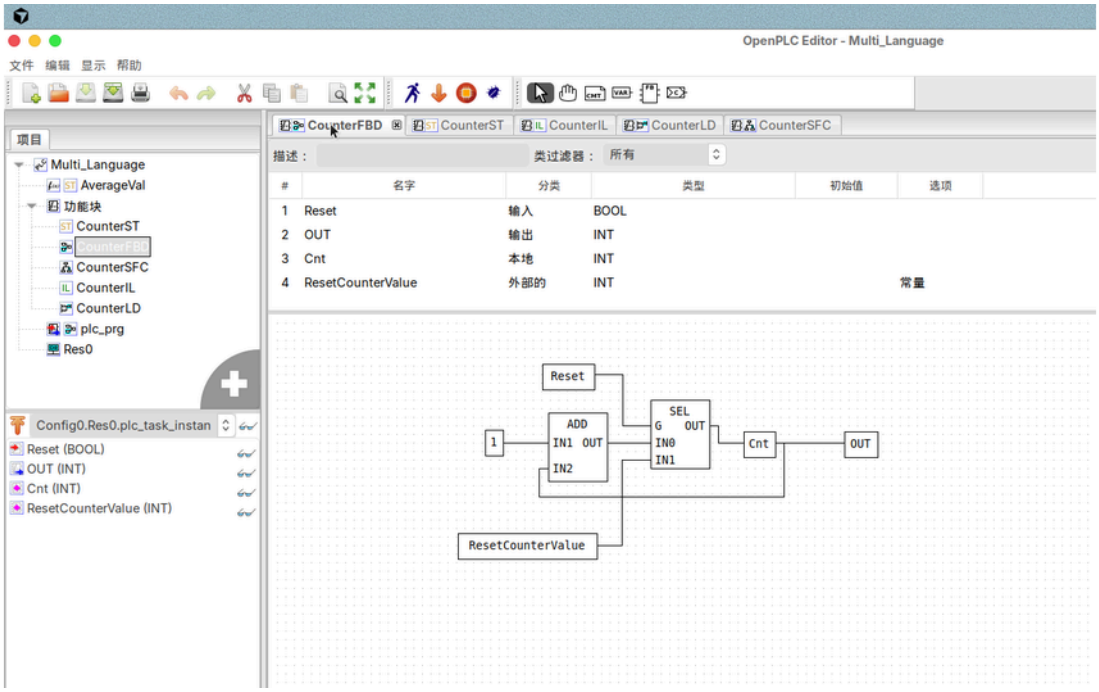
在 LPC Smarteh IDE 中创建配置后，可通过以下方式启动 LPC Manager：

–

–

本软件基于 Beremiz 开源软件，并针对 Smarteh LPC 控制器进行适配，支持 IEC 61131-3 标准编程语言：IL（指令表）、ST（结构化文本）、LD（梯形图）、FBD（功能块图）和 SFC（顺序功能图）。

LPC Manager 软件易于使用，并在 LPC
控制器应用软件的编程、调试、监视与趋势分析方面提供多种功能。



2.2 LPC Manager 编辑器

LPC Manager 软件由以下部分组成：

主菜单：文件、编辑、显示、帮助

工具栏：保存、打印、撤销、重做、剪切、复制、粘贴、在项目中搜索；
选择对象、移动视图、创建新注释、创建新变量、

拓扑与项目窗口

变量与编辑器工作区窗口

搜索、控制台与 PLC 日志窗口

库与调试窗口

2.3 Main menu

在主菜单中可找到文件、编辑、显示

文件、编辑、显示和帮助

2.3.1

保存

关闭标签

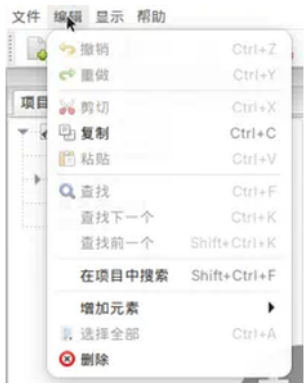
页面设置

打印预览

打印

退出

2.3.2



撤销

重做

Cut

复制

粘贴
- 将（先前复制的）元素粘贴到

查找

查找下一个

查找前一个

在项目中搜索

增加元素

选择全部

删除

2.3.3

刷新

清除错误

显示比例

复位透视图

2.3.4

关于

2.4 Toolbars

工具栏 1

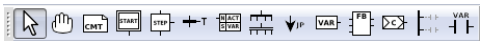
离线模式

仿真模式

应用模式

Bootloader 模式

工具栏 3



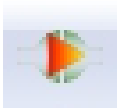
工具栏 2

仿真 PLC

Pressing the “Simulate” button to start PLC emulation running on PC. All debugging functions are supported, same like in online debugging mode while connected to an LPC-2 controller.

将项目构建到 build 文件夹

Press the “Build” button to start building the project.
The “Log Console” displays different building steps. The build results in an executable code, named as the project name. It's located in the build directory of the project.



传输 PLC

This and following commands are only available when PC is connected to the LPC-2 controller using USB programming cable and if blue LED, representing USB connectivity, is on.

按下“传输（Transfer）”按钮可将可执行应用

启动 PLC

This command is active, when operation mode switch on the connected LPC-2 controller is in the “RUN” position. By pressing the “Run” button, the controller will start to execute the application. Green “RUN” LED will switch on.

停止运行中的 PLC

By pressing the “Stop” button, the connected LPC-2 controller will stop all LPC-2 controller processes. The green “RUN” LED and connected modules outputs will go to off state.

工具栏 3

选择对象

Standard tool to select one or more objects inside POU.

移动视图

Move current view inside POU to the desired direction.

适用于 LD、FBD 和 SFC。

创建新注释

Insert a new comment into POU.

适用于 LD、FBD 和 SFC。

创建新电源轨

Insert a power rail (left or right) into POU.

适用于 LD 和 SFC。

创建新线圈

Insert a coil into POU.

适用于 LD。

创建新触点

Insert a contact into POU.

适用于 LD 和 SFC。

创建新变量

Insert a variable into POU.

适用于 LD 和 SFC。

创建新功能块

Insert a block into POU.

适用于 LD、FBD 和 SFC。

创建新连接

Insert a connection into POU.

适用于 LD、FBD 和 SFC。

创建新初始步

Insert an initial step into POU.

适用于 SFC。

创建新步

Insert a new step into POU.

适用于 SFC。

创建新转换

Insert a transition into POU.

适用于 SFC。

创建新动作块

Insert an action block into POU.

适用于 SFC。

创建新分支

Insert a divergence into POU.

适用于 SFC。

创建新跳转

Insert a jump into POU.

适用于 SFC。

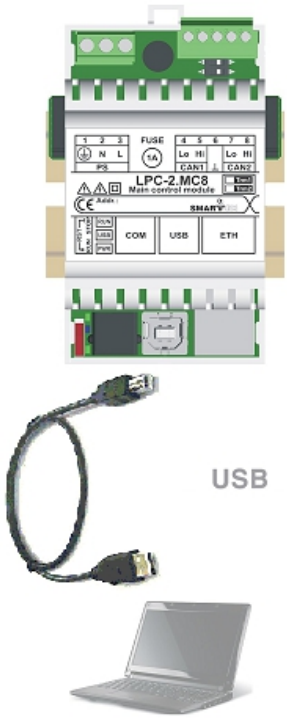
2.5 项目窗口

项目窗口

2.5.1 拓扑

拓扑

构建 - 传输流程



当首次打开 LPC Manager（对应某个 LPC Smarteh IDE 配置）时，工具栏 2 中有两个可用按钮：仿真（Simulate）与构建（Build）。在 POU 中创建所需应用后，必须对该应用进行构建。可在控制台窗口中观察此过程。若代码无错误，则相应文件会生成在项目文件夹中。

下一步是将生成的二进制代码传输到 LPC-2 控制器；控制器须通过 USB 编程电缆连接到 USB 端口。此时 USB 连接（蓝色）LED 点亮，工具栏 2 中会增加“传输”按钮，且“仿真”变为“停止”。单击传输并在 LOG 控制台观察进度。若一切正常，将报告“PLC transferred successfully”，控制器会自动启动（绿色 RUN LED 点亮）。

若向 MCU 传输程序遇到问题，请将 MCU 切换到 STOP 位置（控制器将进入 bootloader 模式，绿色“RUN”LED 应熄灭），然后再次按传输。

重要：传输时，所有内部存储器（保持数据、RTC 等）将被擦除。

2.5.2 项目（程序结构窗口）

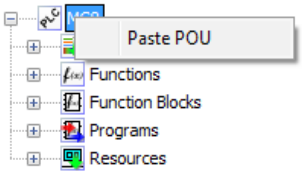
项目（MC8）

添加新项（数据类型、功能、功能块、程序和资源）

新建 POU

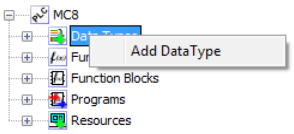
右键菜单（项目名称）

粘贴 POU

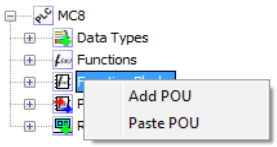


右键菜单（数据类型、功能、功能块、程序和资源）

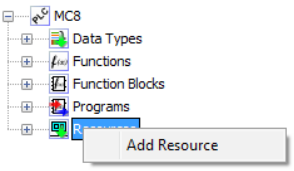
添加数据类型



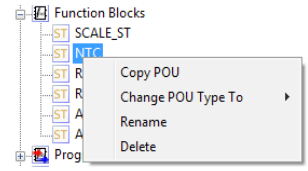
添加 POU

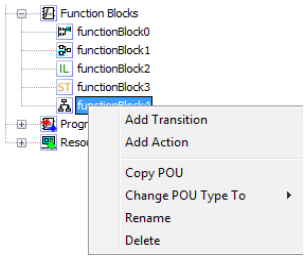


添加资源



右键菜单（功能、功能块和程序）





Contains of the Right-click menu(Function block) depends on a programming language and type of POU.
添加转换
- （仅适用于 SFC 语言）
添加动作

- （仅适用于 SFC 语言）
复制 POU
- 功能块可复制（粘贴）到功能块分区中。

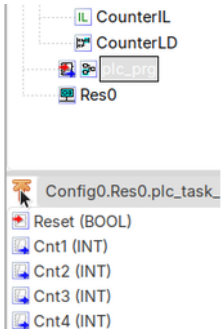
更改 POU 类型为
- 可将所选功能的 POU 类型更改为功能块或程序，

提示：已复制功能块的内容（剪贴板）可保存为文本文件，

双击（项目名称、数据类型、功能、功能块、程序和资源）

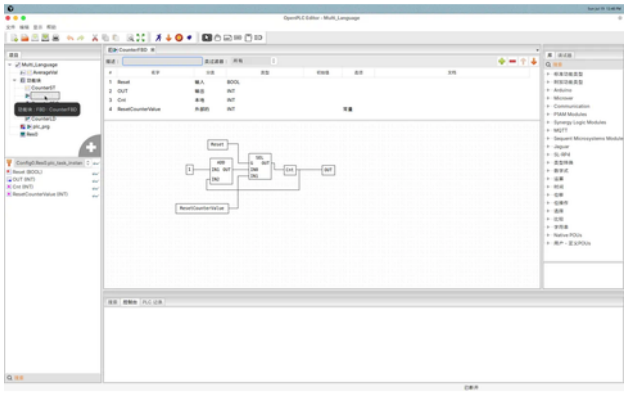
双击项目名称可在编辑器工作区中打开配置变量与项目属性。

2.5.3 项目（实例窗口）



实例

2.5.4 编辑器工作区



编辑器工作区

- Used for editing, setting, programming and debugging of all elements in the project window (POUs, data types, configurations, resources, topology and other variables, instances,...). Edit elements
are opened in
a separate window from those that are listed in the workspace.

快捷键：

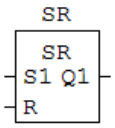
2.6 Library

块属性

可通过双击功能块打开块属性窗口。部分块可在 Inputs 中增加输入（如 ADD）。也可自定义 Execution Order。勾选 Execution Control 将增加 EN/ENO 引脚以动态控制执行。

2.6.1 标准功能块

SR



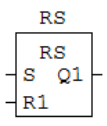
SR 双稳态

SR 双稳态为置位优先的锁存器。

(BOOL:S1, BOOL:R) => (BOOL:Q1)

该函数表示标准的置位优先置位/复位触发器。当输入 S1 为 TRUE 且输入 R 为 FALSE 时，输出 Q1 变为 TRUE。同样，当输入 S1 为 FALSE 且输入 R 为 TRUE 时，输出 Q1 变为 FALSE。在上述任一转换之后，当 S1 与 R 均回到 FALSE 时，输出 Q1 将保持先前状态，直至出现新的条件。若对两个信号同时施加 TRUE，则输出 Q1 被强制为 TRUE (置位优先) 。

RS



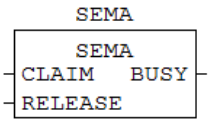
RS 双稳态

RS 双稳态为复位优先的锁存器。

(BOOL:S, BOOL:R1) => (BOOL:Q1)

该函数表示标准的复位优先置位/复位触发器。当输入 S 为 TRUE 且输入 R1 为 FALSE 时，输出 Q1 变为 TRUE。同样，当输入 S 为 FALSE 且输入 R1 为 TRUE 时，输出 Q1 变为 FALSE。在上述任一转换之后，当 S 与 R1 均回到 FALSE 时，输出 Q1 将保持先前状态，直至出现新的条件。若对两个信号同时施加 TRUE，则输出 Q1 被强制为 FALSE (复位优先) 。

SEMA



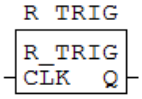
信号量

信号量提供一种机制，使软件元素能够对特定资源进行互斥访问。

(BOOL:CLAIM, BOOL:RELEASE) => (BOOL:BUSY)

该功能块实现信号量功能。通常用于事件同步。BUSY 输出由 CLAIM 输入上的 TRUE 条件激活，并由 RELEASE 输入上的 TRUE 条件去激活。

R TRIG



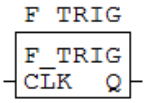
上升沿检测器

当检测到上升沿时，输出产生一个单脉冲。

(BOOL:CLK) => (BOOL:Q)

该函数为上升沿检测器。当在 CLK 输入上检测到 0 到 1 (或 FALSE 到 TRUE，或 OFF 到 ON) 的条件时，输出 Q 变为 TRUE，并在一个完整扫描周期内保持该状态。

F TRIG



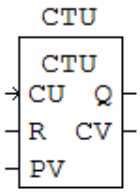
下降沿检测器

当检测到下降沿时，输出 Q 产生一个单脉冲。

(BOOL:CLK) => (BOOL:Q)

该函数为下降沿检测器。当在 CLK 输入上检测到 1 到 0 (或 TRUE 到 FALSE，或 ON 到 OFF) 的条件时，输出 Q 变为 TRUE，并在一个完整扫描周期内保持该状态。

CTU
CTU_DINT,



加计数器

加计数器可用于在计数值达到最大值时发出信号。

CTU: (BOOL:CU, BOOL:R, INT:PV) => (BOOL:Q, INT:CV)

CTU_DINT: (BOOL:CU, BOOL:R, DINT:PV) => (BOOL:Q, DINT:CV)

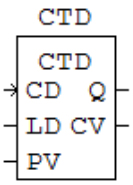
CTU_LINT: (BOOL:CU, BOOL:R, LINT:PV) => (BOOL:Q, LINT:CV)

CTU_UDINT: (BOOL:CU, BOOL:R, UDINT:PV) => (BOOL:Q, UDINT:CV)

CTU_ULINT: (BOOL:CU, BOOL:R, ULINT:PV) => (BOOL:Q,

CTU 函数表示加计数器。CU
输入上的上升沿会使计数器加一。当施加于输入 PV
的设定值到达时，输出 Q 变为 TRUE。在 R 输入上施加 TRUE
信号会将计数器复位为零（异步复位）。CV 输出报告当前计数值。

CTD
CTD_DINT, CTD_LINT,
CTD_UDINT, CTD_ULINT



减计数器

减计数器可用于在从预设值向下计数到零时发出信号。

CTD: (BOOL:CD, BOOL:LD, INT:PV) => (BOOL:Q, INT:CV)

CTD_DINT: (BOOL:CD, BOOL:LD, DINT:PV) => (BOOL:Q, DINT:CV)

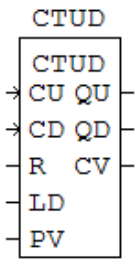
CTD_LINT: (BOOL:CD, BOOL:LD, LINT:PV) => (BOOL:Q, LINT:CV)

CTD_UDINT: (BOOL:CD, BOOL:LD, UDINT:PV) => (BOOL:Q, UDINT:CV)

CTD_ULINT: (BOOL:CD, BOOL:LD, ULINT:PV) => (BOOL:Q,

CTD 函数表示减计数器。CD
输入上的上升沿会使计数器减一。当当前计数值小于或等于零时，输出
Q 变为 TRUE。在 LD（LOAD）输入上施加 TRUE
信号会将计数器装载为输入 PV 上的值（异步装载）。CV
输出报告当前计数值。

CTUD
CTUD_DINT,
CTUD_LINT,
CTUD_UDINT,
CTUD_ULINT



加减计数器

加减计数器有两个输入 CU 与 CD，可分别用于加计数与减计数。

CTUD: (BOOL:CU, BOOL:CD, BOOL:R, BOOL:LD, INT:PV) => (BOOL:QU, BOOL:QD, INT:CV)

CTUD_DINT: (BOOL:CU, BOOL:CD, BOOL:R, BOOL:LD, DINT:PV) => (BOOL:QU, BOOL:QD, DINT:CV)

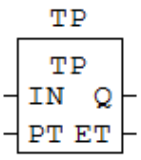
CTUD_LINT: (BOOL:CU, BOOL:CD, BOOL:R, BOOL:LD, LINT:PV) => (BOOL:QU, BOOL:QD, LINT:CV)

CTUD_UDINT: (BOOL:CU, BOOL:CD, BOOL:R, BOOL:LD, UDINT:PV) => (BOOL:QU, BOOL:QD, UDINT:CV)

CTUD_ULINT: (BOOL:CU, BOOL:CD, BOOL:R, BOOL:LD, ULINT:PV) =>

该函数表示可编程加减计数器。CU 输入上升沿加一，CD 输入上升沿减一。R 为 TRUE 时复位为零；LD 为 TRUE 时装载 PV。QU 在计数值≥设定值时有效，QD 在计数值≤0 时有效。CV 报告当前值。

TP



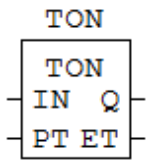
脉冲定时器

脉冲定时器可用于生成给定持续时间的输出脉冲。

(BOOL:IN, TIME:PT) => (BOOL:Q, TIME:ET)

当在 IN 输入上检测到上升沿时，输出 Q 立即变为 TRUE，并持续至设定时间 PT 届满。PT 届满后，若 IN 仍有效则 Q 保持 ON，否则返回 OFF。该定时器不可重触发。ET 报告已经过时间。

TON



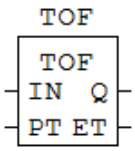
接通延时定时器

接通延时定时器可用于在输入变为真之后，延迟固定时间再将输出置为真。

(BOOL:IN, TIME:PT) => (BOOL:Q, TIME:ET)

将 IN 置为有效即可启动定时器。当 PT 届满且 IN 仍有效时，Q 变为 TRUE，并持续至 IN 释放。若提前释放 IN，定时器清零。ET 报告已经过时间。

TOF



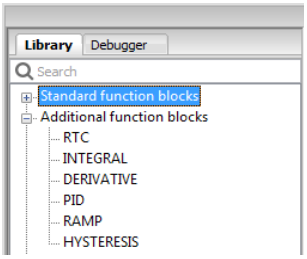
断开延时定时器

断开延时定时器可用于在输入变为假之后，延迟固定时间再将输出置为假。

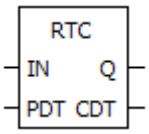
(BOOL:IN, TIME:PT) => (BOOL:Q, TIME:ET)

将 IN 置为有效会立即激活 Q。释放 IN 开始计时；PT 届满且 IN 仍释放时 Q 变为 FALSE。若计时结束前再次置位 IN，定时器清零且 Q 保持 TRUE。ET 报告已经过时间。

2.6.2 附加功能块



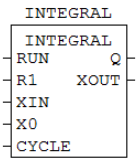
RTC



RTC

Functioning is not supported by our PLC.

INTEGRAL



Integral

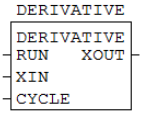
积分功能块对输入 XIN 的值按时间进行积分。

(BOOL:RUN, BOOL:R1, REAL:XIN, REAL:X0, TIME:CYCLE) =>
(BOOL:Q, REAL:XOUT)

当 RUN 为 True 且 R1 为 False 时，XOUT 按 CYCLE 采样随 XIN 变化。当 RUN 为 False 且 R1 为 True 时，XOUT 保持最后输出。若 R1 为 True，XOUT 设为 X0。

XOUT = XOUT + (XIN * CYCLE)

DERIVATIVE



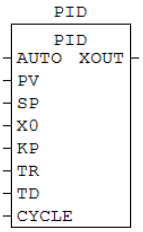
Derivative

微分功能块产生与输入 XIN 变化率成比例的输出 XOUT。

(BOOL:RUN, REAL:XIN, TIME:CYCLE) => (REAL:XOUT)

当 RUN 为 True 时，XOUT 按 XIN 变化率并依据 CYCLE 采样成比例变化。

PID



PID

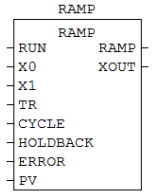
PID
功能块为闭环控制提供经典三项控制器。它不包含输出限制等实际过程控制常用参数（另见 PID_A）。

(BOOL:AUTO, REAL:PV, REAL:SP, REAL:X0, REAL:KP, REAL:TR, REAL:TD, TIME:CYCLE) => (REAL:XOUT)

当 AUTO 为 False 时，XOUT 跟随 X0；为 True 时根据误差（PV-SP）、KP、TR、TD 与 CYCLE 计算。

XOUT = KP * ((PV-SP) + (I_OUT/TR) + (D_OUT * TD))

RAMP



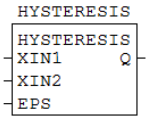
Ramp

RAMP 功能块以标准示例为模型，并增加了 Holdback 功能。

(BOOL:RUN, REAL:X0, REAL:X1, TIME:TR, TIME:CYCLE, BOOL:HOLDBACK, REAL:ERROR, REAL:PV) => (BOOL:RAMP, REAL:XOUT)

当 RUN 与 HOLDBACK 均为 False 时，XOUT 跟随 X0；当 RUN 为 True 且 HOLDBACK 为 False 时，XOUT 每个 CYCLE 按 OUT(to-1)+(X1-XOUT(to-1)) 变化。

HYSTERESIS



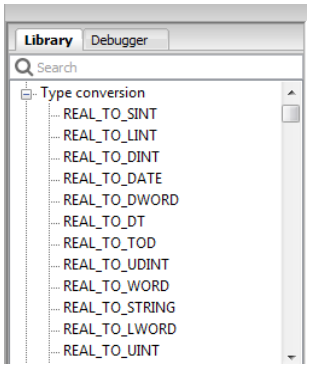
Hysteresis

滞环功能块根据 XIN1 与 XIN2 的差值提供滞环布尔输出。

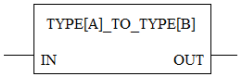
(REAL:XIN1, REAL:XIN2, REAL:EPS) => (BOOL:Q)

当 XIN1 > XIN2+EPS 时 Q 为 True；当 XIN1 < XIN2-EPS 时 Q 为 False。

2.6.3 类型转换



TYPE[A]_TO_TYPE[B]

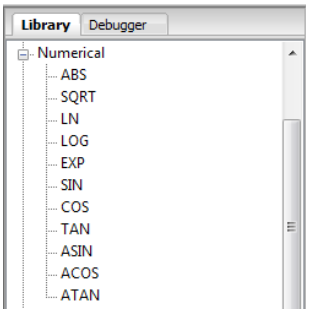


Data type conversion

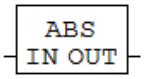
(TYPE[A]:IN) => (TYPE[B]:OUT)

ST 语法示例：

2.6.4 Numerical



ABS

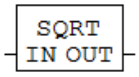


Absolute number

(ANY_NUM:IN) => (ANY_NUM:OUT)

ST 语法示例：

SQRT

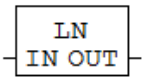


Square root (base 2)

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

LN

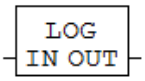


Natural logarithm

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

LOG

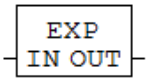


Logarithm to base 10

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

EXP

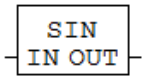


Exponentiation

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

SIN

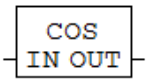


Sine

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

COS

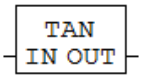


Cosine

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

TAN

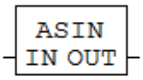


Tangent

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

ASIN

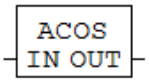


Arc sine

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

ACOS

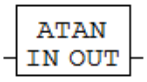


Arc cosine

(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

ATAN

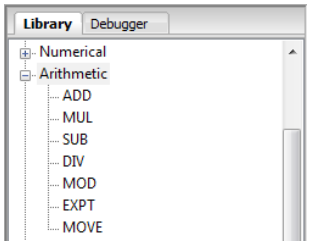


Arc tangent

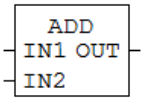
(ANY_REAL:IN) => (ANY_REAL:OUT)

ST 语法示例：

2.6.5 Arithmetic



ADD



Addition

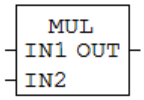
(ANY_NUM:IN1, ANY_NUM:IN2) => (ANY_NUM:OUT)

OUT = IN1 + IN2.

输入数量可扩展。

ST 语法示例：

MUL



Multiplication

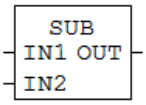
(ANY_NUM:IN1, ANY_NUM:IN2) => (ANY_NUM:OUT)

OUT = IN1 * IN2.

输入数量可扩展。

ST 语法示例：

SUB



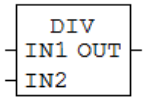
Subtraction

(ANY_NUM:IN1, ANY_NUM:IN2) => (ANY_NUM:OUT)

OUT = IN1 – IN2.

ST 语法示例：

DIV



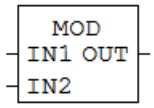
Division

(ANY_NUM:IN1, ANY_NUM:IN2) => (ANY_NUM:OUT)

OUT = IN1 / IN2.
For example 1234 / 10 = 3.

ST 语法示例：

MOD



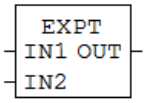
Remainder (modulo)

(ANY_NUM:IN1, ANY_NUM:IN2) => (ANY_NUM:OUT)

OUT = IN1 modulo IN2.
For example 1234 modulo 10 = 4.

ST 语法示例：

EXPT



Exponent

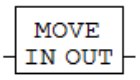
(ANY_REAL:IN1, ANY_NUM:IN2) => (ANY_REAL:OUT)

OUT = IN1 IN2.

例如 23= 8.

ST 语法示例：

MOVE



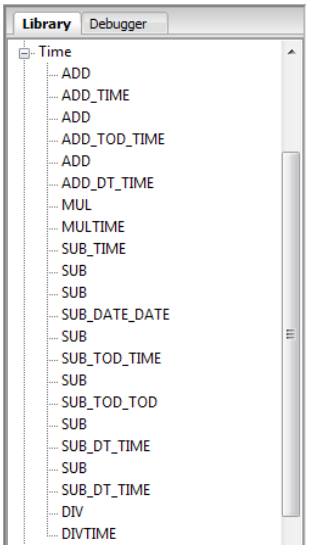
Assignment

(ANY:IN) => (ANY:OUT)

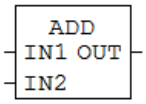
OUT = IN.

ST 语法示例：

2.6.6 Time



ADD

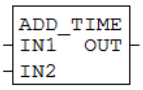


Time addition

(TIME:IN1, TIME:IN2) => (TIME:OUT)

输入数量可扩展。

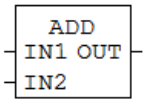
ADD_TIME



Time addition

(TIME:IN1, TIME:IN2) => (TIME:OUT)

ADD

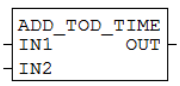


Time-of-day addition

(TOD:IN1, TIME:IN2) => (TOD:OUT)

输入数量可扩展。

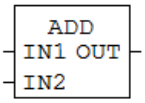
ADD_TOD_TIME



Time-of-day addition

(TOD:IN1, TIME:IN2) => (TOD:OUT)

ADD

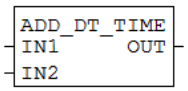


Addition

(ANY_NUM:IN1, ANY_NUM:IN2) => (ANY_NUM:OUT)

输入数量可扩展。

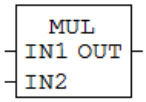
ADD_DT_TIME



日期

(DT:IN1, TIME:IN2) => (DT:OUT)

MUL

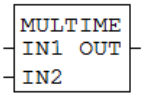


Multiplication

(ANY_NUM:IN1, ANY_NUM:IN2) => (ANY_NUM:OUT)

输入数量可扩展。

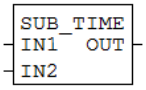
MULTIME



Time multiplication

(TIME:IN1, ANY_NUM:IN2) => (TIME:OUT)

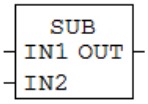
SUB_TIME



Time subtraction

(TIME:IN1, TIME:IN2) => (TIME:OUT)

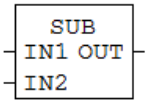
SUB



Time subtraction

(TIME:IN1, TIME:IN2) => (TIME:OUT)

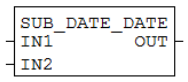
SUB



日期

(DATE:IN1, DATE:IN2) => (TIME:OUT)

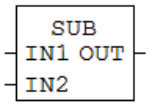
SUB_DATE_DATE



日期

(DATE:IN1, DATE:IN2) => (TIME:OUT)

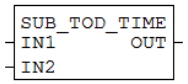
SUB



Time-of-day subtraction

(TOD:IN1, TIME:IN2) => (TOD:OUT)

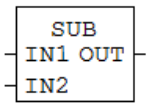
SUB_TOD_TIME



Time-of-day subtraction

(TOD:IN1, TIME:IN2) => (TOD:OUT)

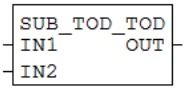
SUB



Time-of-day subtraction

(TOD:IN1, TOD:IN2) => (TIME:OUT)

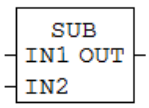
SUB_TOD_TOD



Time-of-day subtraction

(TOD:IN1, TOD:IN2) => (TIME:OUT)

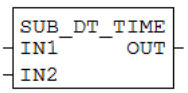
SUB



日期

(DT:IN1, TIME:IN2) => (DT:OUT)

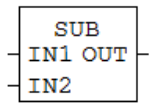
SUB_DT_TIME



日期

(DT:IN1, TIME:IN2) => (DT:OUT)

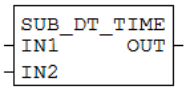
SUB



日期

(DT:IN1, DT:IN2) => (TIME:OUT)

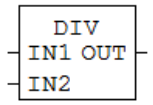
SUB_DT_TIME



日期

(DT:IN1, DT:IN2) => (TIME:OUT)

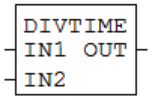
DIV



Time division

(TIME:IN1, ANY_NUM:IN2) => (TIME:OUT)

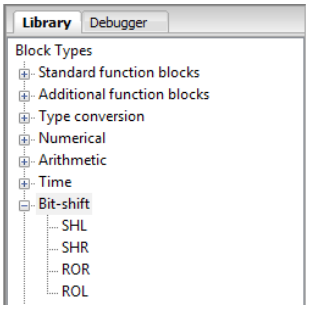
DIVTIME



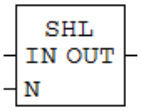
Time division

(TIME:IN1, ANY_NUM:IN2) => (TIME:OUT)

2.6.7 Bit-shift



SHL



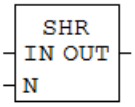
Shift left

(ANY_BIT:IN, ANY_INT:N) => (ANY_BIT:OUT)

OUT represents IN variable shifted left by N bits. Zeros are filled on the right side of the OUT variable.

ST 语法示例：

SHR



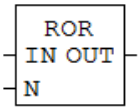
Shift right

(ANY_BIT:IN, ANY_INT:N) => (ANY_BIT:OUT)

OUT represents IN variable shifted right by N bits. Zeros are filled on the left side of the OUT variable.

ST 语法示例：

ROR



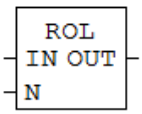
Rotate right

(ANY_NBIT:IN, ANY_INT:N) => (ANY_NBIT:OUT)

OUT represents IN variable right rotated by N bits. Each rotation most right bit is filled into most left bit of the OUT variable.

ST 语法示例：

ROL



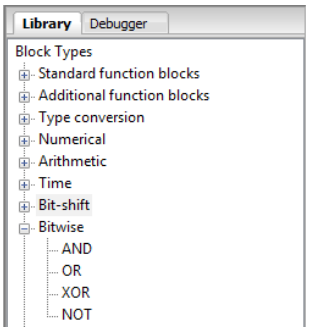
Rotate left

(ANY_NBIT:IN, ANY_INT:N) => (ANY_NBIT:OUT)

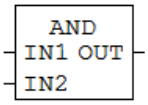
OUT represents IN variable left rotated by N bits. Each rotation most left bit is filled into most right bit of the OUT variable.

ST 语法示例：

2.6.8 Bitwise



AND



按位运算

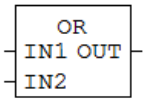
(ANY_BIT:IN1, ANY_BIT:IN2) => (ANY_BIT:OUT)

OUT = IN1 AND IN2.

输入数量可扩展。

ST 语法示例：

OR



按位运算

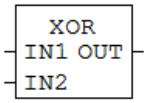
(ANY_BIT:IN1, ANY_BIT:IN2) => (ANY_BIT:OUT)

OUT = IN1 OR IN2.

输入数量可扩展。

ST 语法示例：

XOR



按位运算

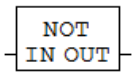
(ANY_BIT:IN1, ANY_BIT:IN2) => (ANY_BIT:OUT)

OUT = IN1 EXCLUSIVE OR IN2.

输入数量可扩展。

ST 语法示例：

NOT



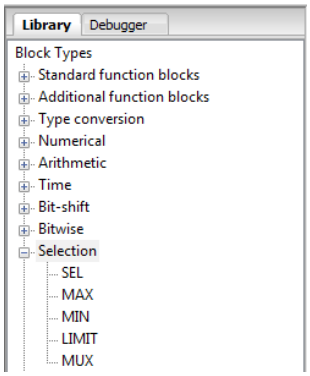
按位运算

(ANY_BIT:IN) => (ANY_BIT:OUT)

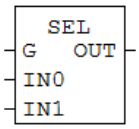
OUT = NOT IN.

ST 语法示例：

2.6.9 Selection



SEL

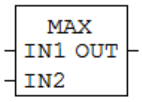


Binary selection (1 of 2)

(BOOL:G, ANY:IN0, ANY:IN1) => (ANY:OUT)

If G is False, OUT will follow IN0 value. If G is True, OUT will follow IN1 value.

MAX



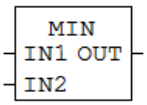
Maximum

(ANY:IN1, ANY:IN2) => (ANY:OUT)

This function block compares the magnitude of the values present at input IN1 and IN2 reporting on its output the largest value (maximum value).

输入数量可扩展。

MIN



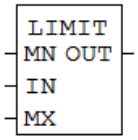
Minimum

(ANY:IN1, ANY:IN2) => (ANY:OUT)

This function block compares the magnitude of the values present at input IN1 and IN2 reporting on its output the smallest value (minimum value).

输入数量可扩展。

LIMIT

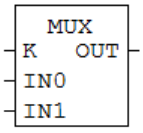


Limitation

(ANY:MN, ANY:IN, ANY:MX) => (ANY:OUT)

OUT will follow IN value between minimum MN and maximum MX value. If IN will be less than minimum MN value, OUT will represent MN value and if IN will be grater than maximum MX value, OUT will represent MX value.

MUX



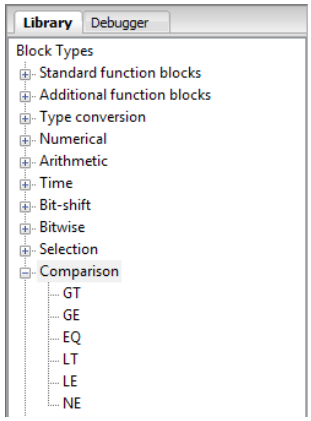
Multiplexer (select 1 of N)

(ANY_INT:K, ANY:IN0, ANY:IN1) => (ANY:OUT)

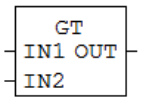
Depends on K value, one of IN1, IN2 .. INn is selected and OUT will represent the selected input value.

输入数量可扩展。

2.6.10 Comparison



GT



Greater than

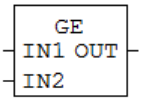
(ANY:IN1, ANY:IN2) => (BOOL:OUT)

OUT will become True if IN1 is greater than IN2, else OUT will be False.

输入数量可扩展。

ST 语法示例：

GE



Greater than or equal to

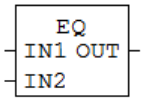
(ANY:IN1, ANY:IN2) => (BOOL:OUT)

OUT will become True if IN1 is greater or equal than IN2, else OUT will be False.

输入数量可扩展。

ST 语法示例：

EQ



Equal to

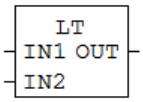
(ANY:IN1, ANY:IN2) => (BOOL:OUT)

OUT will become True if IN1 and IN2 are equal, else OUT will be False.

输入数量可扩展。

ST 语法示例：

LT



Less than

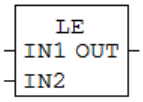
(ANY:IN1, ANY:IN2) => (BOOL:OUT)

OUT will become True if IN1 is less than IN2, else OUT will be False.

输入数量可扩展。

ST 语法示例：

LE



Less than or equal to

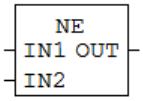
(ANY:IN1, ANY:IN2) => (BOOL:OUT)

OUT will become True if IN1 is less or equal than IN2, else OUT will be False.

输入数量可扩展。

ST 语法示例：

NE



Not equal to

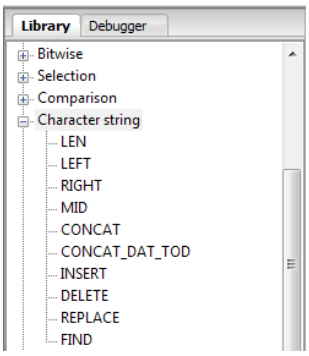
(ANY:IN1, ANY:IN2) => (BOOL:OUT)

OUT will become True if IN1 and IN2 are NOT equal, else OUT will be False.

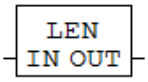
输入数量可扩展。

ST 语法示例：

2.6.11 字符串



LEN

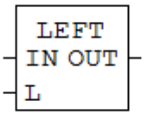


Length of string

(STRING:IN) => (INT:OUT)

OUT represents number of characters in IN string. 例如 IN string is 'ABCDEFGH', OUT will be 8.

LEFT

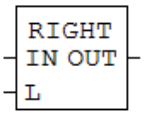


String left of

(STRING:IN, ANY_INT:L) => (STRING:OUT)

OUT represents string of L number of characters, leftmost of IN string. 例如 IN string is 'ABCDEFGH', L is 3, OUT string will be 'ABC'.

RIGHT

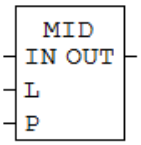


String right of

(STRING:IN, ANY_INT:L) => (STRING:OUT)

OUT represents string of L number of characters, rightmost of IN string. 例如 IN string is 'ABCDEFGH', L is 3, OUT string will be 'FGH'.

MID

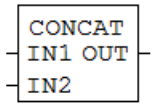


String from the middle

(STRING:IN, ANY_INT:L, ANY_INT:P) => (STRING:OUT)

OUT represents string of L characters of string IN, beginning at position P from left side of IN string. 例如 IN string is 'ABCDEFGH', L is 4 and P is 2. OUT string will be 'DE'.

CONCAT



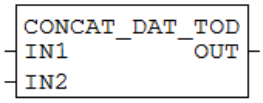
Concatenation

(STRING:IN1, STRING:IN2) => (STRING:OUT)

OUT represents concatenated string of IN1 and IN2. For example IN1 string is 'ABCD' and IN2 string is 'EFG', OUT string will be 'ABCDEFG'.

输入数量可扩展。

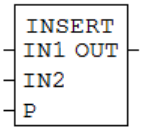
CONCAT_DAT_TOD



Time concatenation

(DATE:IN1, TOD:IN2) => (DT:OUT)

INSERT

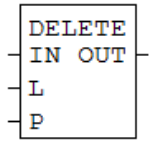


Insertion (into)

(STRING:IN1, STRING:IN2, ANY_INT:P) => (STRING:OUT)

OUT represents inserted string of IN2 to the IN1 string, after P position from left side of string IN1. 例如 if P is 2, IN1 string is 'ABC' and IN2 string is '12', OUT string will be 'AB12C'.

DELETE

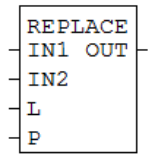


Deletion (within)

(STRING:IN, ANY_INT:L, ANY_INT:P) => (STRING:OUT)

OUT represents deleted string for number of L characters, beginning at IN string P position from left side of string IN1. For example L is 3, P is 2 and IN2 string is 'ABCDEFG, OUT string will be 'AEFG'.

REPLACE

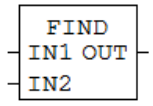


Replacement (within)

(STRING:IN1, STRING:IN2, ANY_INT:L, ANY_INT:P) =>
(STRING:OUT)

OUT represents replaced string for number of L characters, beginning at P position from left side of string IN1. 例如 L is 3, P is 2 and IN2 string is 'ABCDEFG, OUT string will be 'AEFG'.

FIND

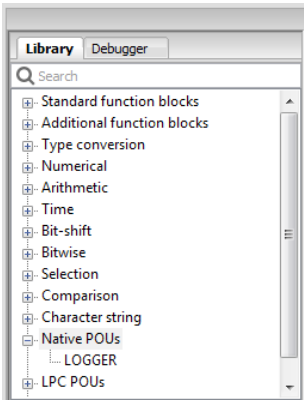


查找

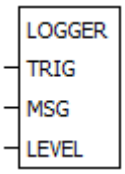
(STRING:IN1, STRING:IN2) => (INT:OUT)

OUT represents first left position in string IN1 where string IN2 starts. If string IN2 is not found in string IN1, OUT will be 0. For example string IN1 is 'ABCDEFG', string IN2 is 'DEF', OUT will be 4.

2.6.12 Native POUs



LOGGER

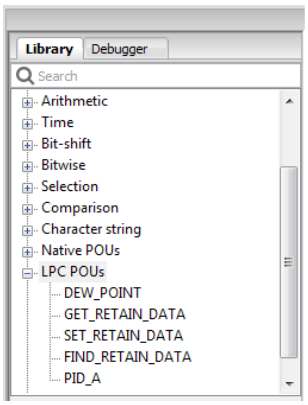


Logger

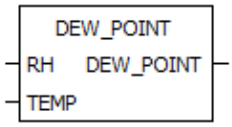
(BOOL:TRIG, STRING:MSG, LOGLEVEL:LEVEL) => ()

Logger is used for off-line logging. Log data defined by trig, message and log level are saved in MCU database. Log data can be read from PLC Log (2.10 PLC Log). Level must be a number between 0 and 3 written as Expression.

2.6.13 LPC POUs



DEW_POINT

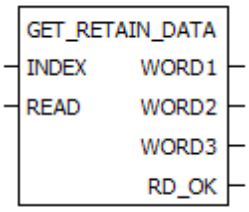


Dew-point

(REAL:RH, REAL:TEMP) => (REAL:DEW_POINT)

Calculate dew-point from temperature and humidity.

GET_RETAIN_DATA



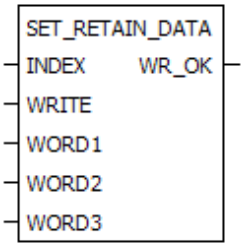
Get retain data

(UINT:INDEX, BOOL:READ) => (UINT:WORD1, UINT:WORD2, UINT:WORD3, BOOL:RD_OK)

Get three variables (WORD1, WORD2 and WORD3) from fixed location (INDEX) inside retain database when READ is on. INDEX can be from 0 to 1999.

Primary used for RFID key card data.

SET_RETAIN_DATA



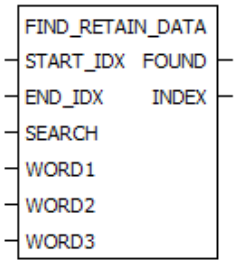
Set retain data

(UINT:INDEX, BOOL:WRITE, UINT:WORD1, UINT:WORD2, UINT:WORD3) => (BOOL:WR_OK)

Set three variables (WORD1, WORD2 and WORD3) to fixed location (INDEX) inside retain database when WRITE is on. INDEX can be from 0 to 1999.

Primary used for RFID key card data.

FIND_RETAIN_DATA



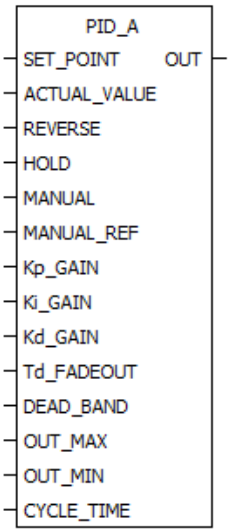
查找

(UINT:START_IDX, UINT:END_IDX, BOOL:SEARCH, UINT:WORD1, UINT:WORD2, UINT:WORD3) => (BOOL:FOUND, UINT:INDEX)

搜索

Primary used for RFID key card data.

PID_A



PID A version

(REAL:SET_POINT, REAL:ACTUAL_VALUE, BOOL:REVERSE, BOOL:HOLD, BOOL:MANUAL, REAL:MANUAL_REF, REAL:Kp_GAIN, REAL:Ki_GAIN, REAL:Kd_GAIN, TIME:Td_FADEOUT, REAL:DEAD_BAND, REAL:OUT_MAX, REAL:OUT_MIN, TIME:CYCLE_TIME) => (REAL:OUT)

PID A version contains most used parameters for automation PID process control usage. Kp, Ki and Kd are independent.

SET_POINT – set-point value

ACTUAL_VALUE – actual value

REVERSE – reverse (forward) mode calculation setting

HOLD – hold mode

MANUAL – manual mode

MANUAL_REF – OUT reference at manual mode

Kp_GAIN – proportional gain parameter

Ki_GAIN – Integral gain parameter

Kd_GAIN – Derivative gain parameter

Td_FADEOUT – fadeout time of Kd_GAIN output influences

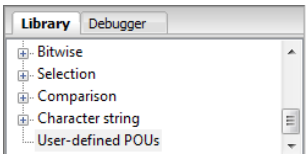
DEAD_BAND – dead band

OUT_MAX - output limitation (maximum)

OUT_MIN - output limitation (minimum)

CYCLE_TIME – calculation cycle time

2.6.14 用户自定义 POU



用户自定义 POU

2.7 Debugger

变量实际值可呈现为：

数值趋势在可选时间范围（10ms…24h）内显示，该范围对所有观察变量通用。

变量也可导出到剪贴板。

也可通过单击挂锁图标强制或释放这些变量。

2.8 Search

搜索

2.9 Console

控制台

2.10 PLC Log

PLC 日志

3 编程语言

LPC Manager 基于 PLC 编程语言国际标准 IEC 61131-3。

可使用以下类型的 PLC 编程语言：

文本类：

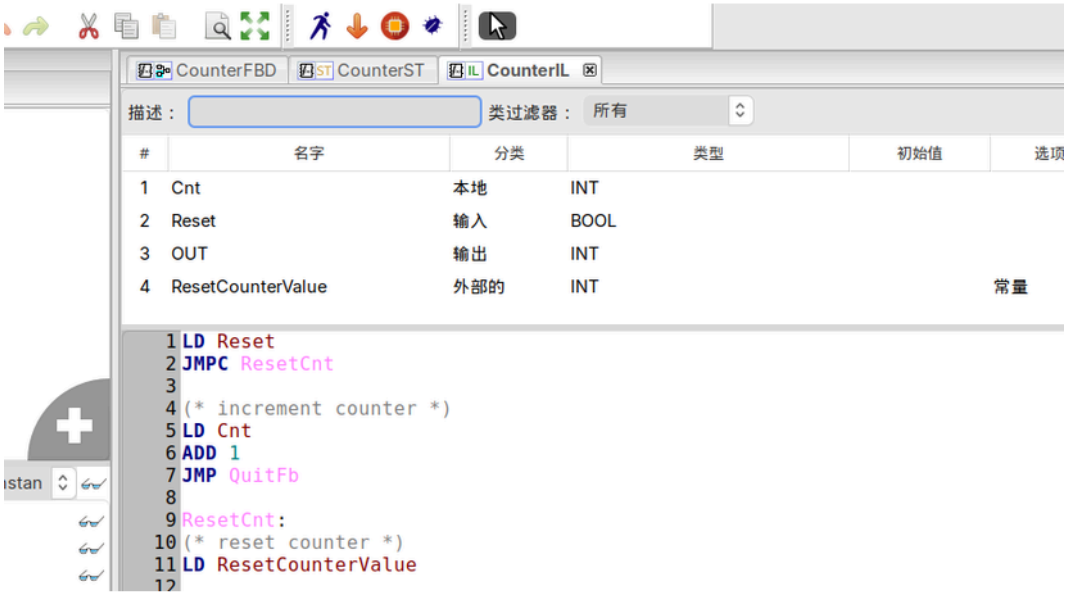
- IL
- ST

图形类：

- FBD
- LD
- SFC

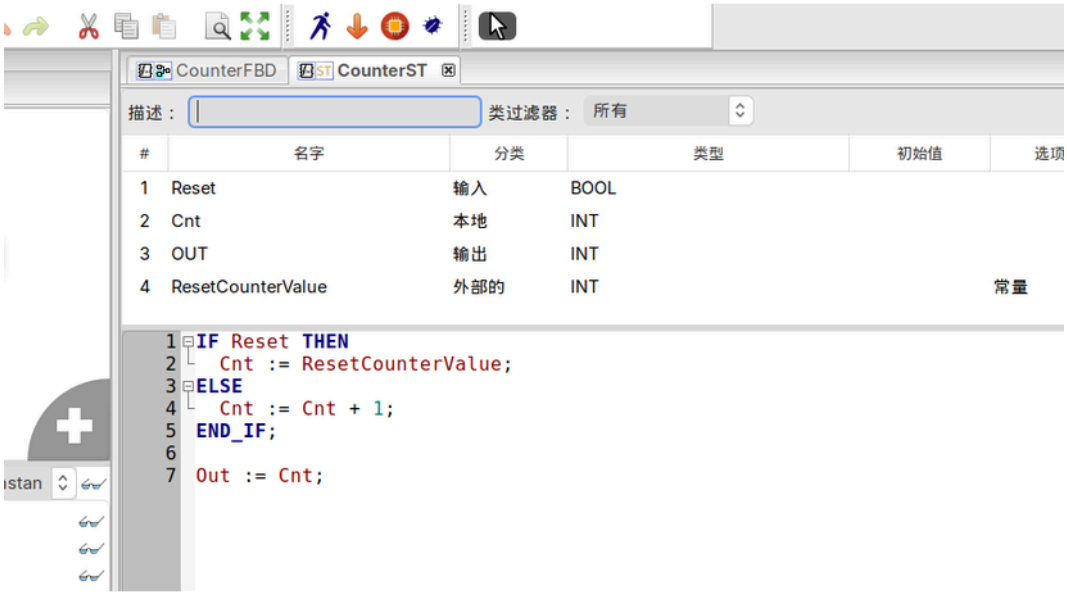
3.1 IL - 指令表

指令表由一系列指令组成，类似于汇编。每条指令另起一行，包含操作符及可选修饰符与操作数。

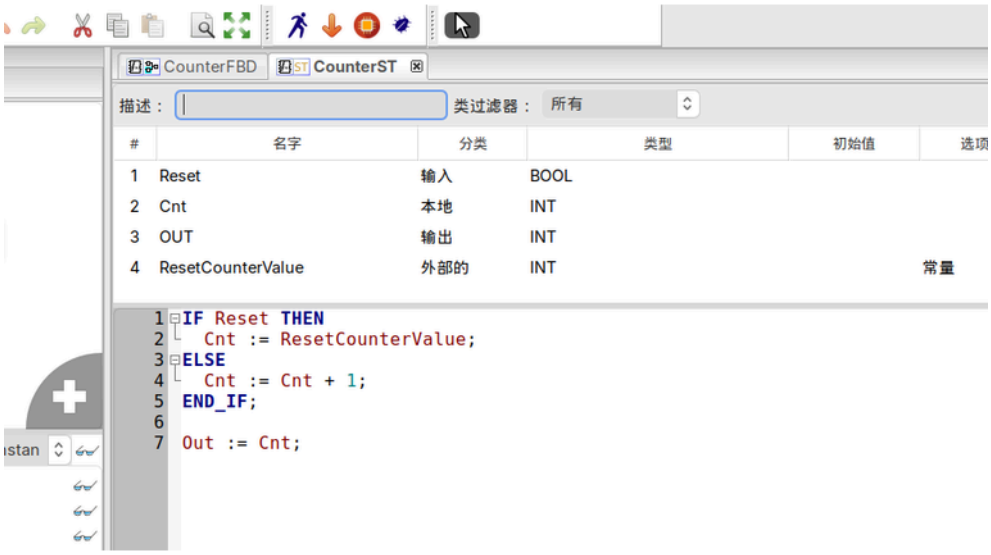


3.2 ST - 结构化文本

结构化文本由一系列指令组成，是类似 C 的高级语言。行尾与空格同等对待。



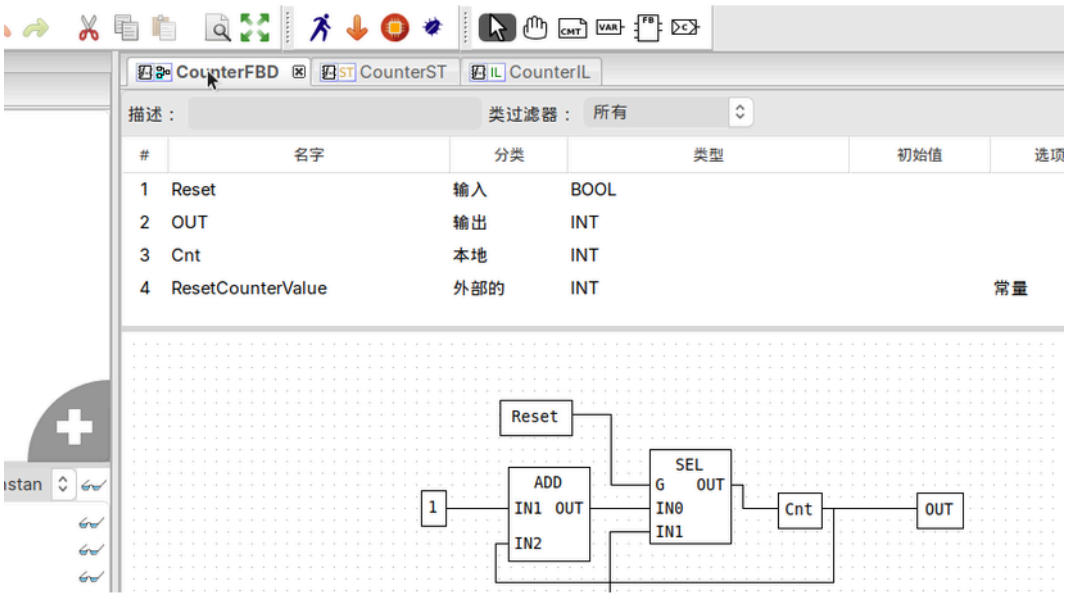
Structured text syntax examples:



3.3 FBD - 功能块图

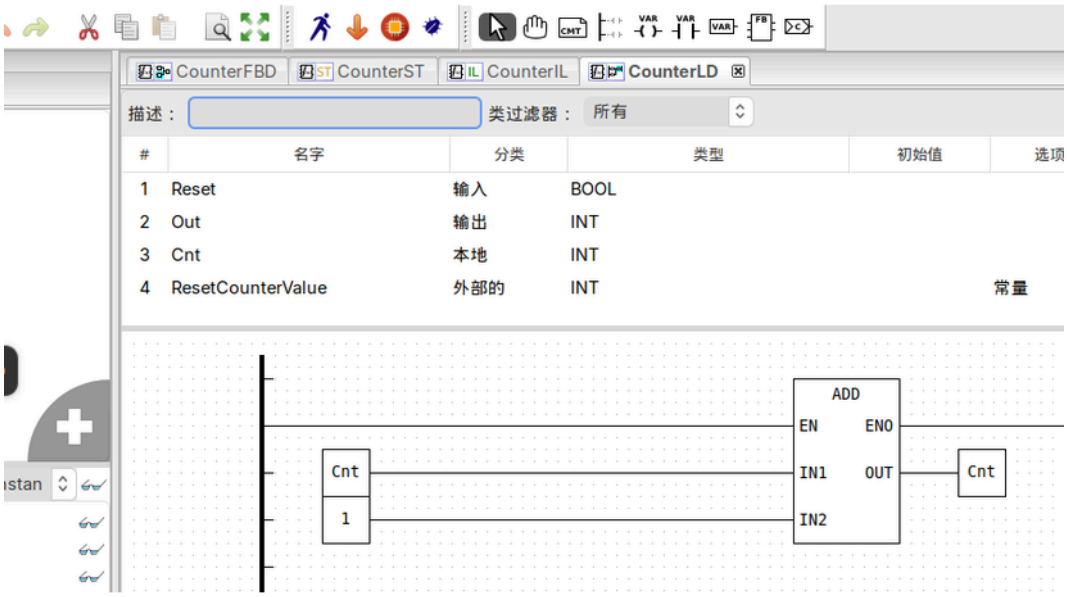
功能块图是一种用于可编程控制器编程的图形化语言。

FBD 元素由信号流线互连。功能块输出不得互连；不允许 LD 的线或，需使用显式 OR 块。



3.4 LD - 梯形图

梯形图是一种图形化语言，以类似继电器梯形逻辑梯级的方式布置符号，左右以电源轨为界。

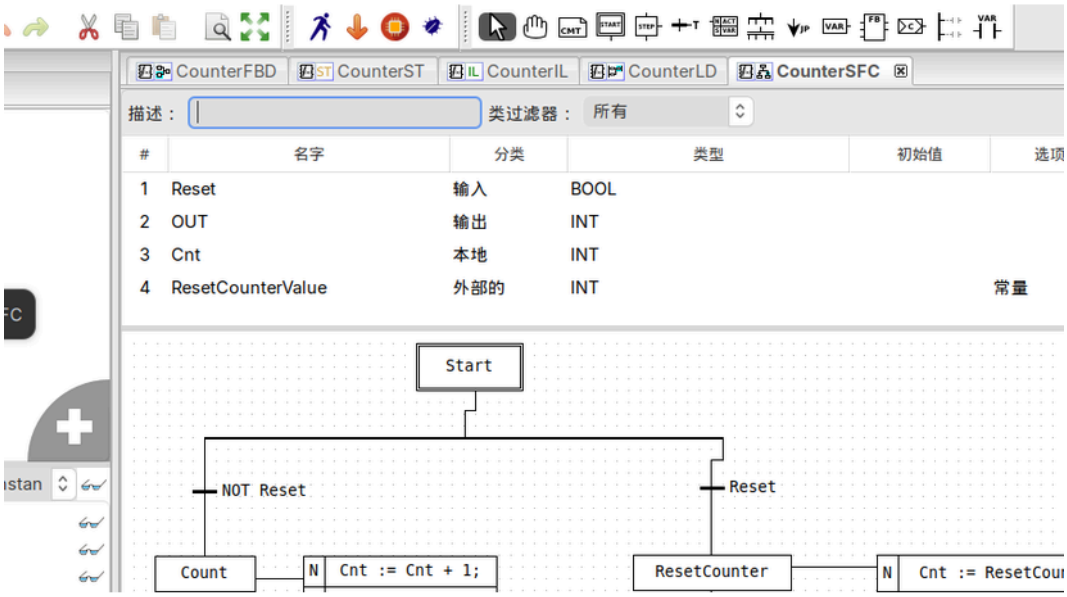


3.5 SFC - 顺序功能图

顺序功能图是一种图形化语言，用于将程序组织单元划分为步与转换，以实现顺序控制。

SFC 元素将程序组织单元划分为由有向连接线互连的步与转换；步关联动作，转换关联条件。

由于需要保存状态，仅功能块与程序可用 SFC 结构化；若部分使用 SFC，则整个 POU 均应如此划分。



附录 A – 错误报告

如果您发现缺陷或有改进想法，欢迎联系 support@smarteh.si。我们将评估并尽量纳入下一版本。

请联系供应商并提供说明，应包含：

若需更多信息我们可能联系您。请记住：唯一不需要维护的软件，是从未被使用的软件！

附录 B – 文档历史

The following table describes all the changes to the document.

日期	
30.09.2011	
30.01.2012	
30.06.2012	
25.05.2014	
22.01.2016	